

6. เหตุใด Two-Pointer จึงใช้ได้เมื่ออาร์เรย์เรียงแล้ว

เพราะอาร์เรย์ถูกเรียงจากน้อยไปมาก ทำให้สามารถตัดสินใจได้ว่า

- ถ้าผลรวมน้อยเกินไป ให้เลื่อน **left** ไปทางขวาเพื่อเพิ่มผลรวม
- ถ้าผลรวมมากเกินไป ให้เลื่อน **right** มาทางซ้ายเพื่อลดผลรวม

จึงค้นหาได้ภายใน $O(n)$

7. หากใช้อาร์เรย์ที่ยังไม่เรียงลำดับจะเกิดอะไรขึ้น

วิธี **Two-Pointer** จะทำงานไม่ถูกต้อง เพราะการเลื่อนตัวชี้ซ้ายหรือขวาจะไม่สามารถรับประกันได้ว่าผลรวมจะเพิ่มหรือลดตามที่ต้องการ จึงอาจไม่พบคำตอบหรือได้ผลลัพธ์ผิด

8. ข้อดีและข้อจำกัดของแต่ละอัลกอริทึม

Brute Force

ข้อดี

- เข้าใจง่าย
- ใช้ได้กับข้อมูลทุกแบบ

ข้อจำกัด

- ช้าที่สุด
- ต้องตรวจสอบทุกคู่

Recursive Two-Pointer

ข้อดี

- เร็วที่สุด

- ตรวจสอบเพียงรอบเดียว
- เหมาะกับข้อมูลที่เรียงแล้ว

ข้อจำกัด

- ใช้ได้เฉพาะอาร์เรย์ที่เรียงลำดับแล้ว
- ใช้ Stack ของ Recursion

Binary Search

ข้อดี

- เร็วกว่า Brute Force
- ใช้ Binary Search ได้อย่างมีประสิทธิภาพ

ข้อจำกัด

- ต้องใช้อาร์เรย์ที่เรียงลำดับแล้ว
- ยังช้ากว่า Two-Pointer

9. สรุป

สำหรับอาร์เรย์ที่เรียงลำดับแล้ว **Recursive Two-Pointer** เป็นอัลกอริทึมที่เหมาะสมที่สุด เพราะมี **Time Complexity** เท่ากับ $O(n)$ ซึ่งเร็วกว่า **Binary Search** ($O(n \log n)$) และ **Brute Force** ($O(n^2)$) อย่างไรก็ตาม วิธี Two-Pointer ใช้งานได้เฉพาะกับข้อมูลที่เรียงลำดับแล้วเท่านั้น ส่วน **Brute Force** สามารถใช้ได้กับข้อมูลทุกประเภทแต่ใช้เวลามากที่สุด ขณะที่ **Binary Search** เป็นทางเลือกที่มีประสิทธิภาพปานกลางเมื่อข้อมูลเรียงลำดับแล้ว.